

Lecture 6

Coupling (Principles, Part 2)

Example Code and Exercises

<https://github.com/eecs498-software-design/lecture>



Kinds of Coupling

- In this course, we'll take a pretty broad view of coupling.
- Two things are coupled when they:
 - Depend on each other (may be one-way or two-way dependence)
 - Need to change together
 - Rely on some shared knowledge, implementation detail, or design decision



Kinds of Coupling

Coupling comes in many different flavors:

- **Mechanical** Dependence on the underlying environment
- **Structural** Dependence on availability of data and operations
- **Behavioral** Dependence on the requirements, results, side effects, or timing of an operation
- **Semantic** Dependence on the meaning or interpretation of data or operations

Coupling may be **explicit**, like literally calling a function and using its result..

Coupling may be **implicit**, like assuming some part of the code left the right data in a variable for us to find later.

The way that abstractions will need to connect to and depend on each other is equally essential as considering the abstractions themselves.



Exercise: Identify Coupling

```
class MinesweeperGame {
  constructor(width: number, height: number, numMines: number) {
    this.width = width;
    this.height = height;
    this.numMines = numMines;
    this.board = /* create new board with random
    this.state = "playing";
    this.cellsRevealed = 0;
  }

  public reset(): void {
    this.board = /* create new board with random
    this.state = "playing";
    this.cellsRevealed = 0;
  }
}
```

```
class MinesweeperGame {
  constructor(width: number, height: number, numMines: number) {
    this.width = width;
    this.height = height;
    this.numMines = numMines;
    this.initializeGameState();
  }

  public reset(): void {
    this.initializeGameState(); // reuses the same logic
  }

  private initializeGameState(): void {
    this.board = /* create new board with random mines */;
    this.state = "playing";
    this.cellsRevealed = 0;
  }
}
```

Exercise: Identify Connections

```
async function main() {  
  const args = process.argv.slice(2);  
  const [width, height, numMines] = /* parse command-line args */;  
  const game = new MinesweeperGame(width, height, numMines);  
  await game.play();  
}  
  
class MinesweeperGame {  
  constructor(width: number, height: number, numMines: number) {  
    // initialize game state  
  }  
}
```



Exercise: Identify Connections

```
async function main() {  
  const args = process.argv.slice(2);  
  const [width, height, numMines] = /* parse command-line args */;  
  const game = new MinesweeperGame(width, height, numMines);  
  await game.play();  
}
```

```
class MinesweeperGame {  
  constructor(width: number, height: number) {  
    // initialize game state  
  }  
}
```

```
async function main() {  
  const game = new MinesweeperGame(width, height, numMines);  
  await game.play();  
}  
  
class MinesweeperGame {  
  constructor(width: number, height: number, numMines: number) {  
    const args = process.argv.slice(2);  
    const [width, height, numMines] = /* parse command-line args */;  
    // initialize game state  
  }  
}
```



Exercise: Identify Connections

```
async function main() {  
  const [width, height, numMines] = /* parse command-line args */;  
  const game = new MinesweeperGame();  
  game.setDimensions(width, height);  
  game.placeMines(numMines);  
  game.playGame();  
}
```



Impact of Coupling

The impact of coupling is modulated by:

- **Complexity** How complicated are the dependencies or shared knowledge?
- **Distance** How far apart are the coupled pieces, both in terms of code and developers?
- **Feedback** How reliably and quickly do we get feedback if there's a breaking change?
- **Volatility** How likely it is that breaking change will occur?

These combine multiplicatively. Aim to ensure at least one is close to 0.

$\text{complexity} * \text{distance} * \text{feedback} * \text{volatility}$

*This framework builds on the work of Vladimir Khononov in *Balancing Coupling in Software Design: Universal Design Principles for Architecting Modular Software Systems*